ExperimentIQ

ML-Powered A/B Testing & Business Experiment Intelligence Platform

Complete Project Blueprint

From problem definition → statistics → machine learning → deep learning → business recommendation → deployment

Project focus	Machine Learning + Statistics + Deep Learning
Domain	Business Analytics / Product Analytics / Experimentation
Primary goal	Predict experiment outcomes and quantify business impact
Suggested stack	Python, Pandas, NumPy, SciPy, Scikit-learn, XGBoost, PyTorch/TensorFlow, Streamlit
Deployment	Streamlit dashboard + optional FastAPI API

Recommended project name: ExperimentIQ

Alternative names: A/B Insight, ExperimentPulse, TestWise, OptiTest ML.

1. Project Overview

ExperimentIQ is an end-to-end machine-learning and statistical experimentation platform. It helps a business analyze an A/B test, estimate conversion and revenue impact, predict the likely winning variant, and turn the result into a business recommendation.

The real-world problem

Companies frequently compare an existing product experience (Variant A) with a new experience (Variant B): a landing page, checkout flow, recommendation strategy, pricing message, advertisement, email subject line, or feature.

A conventional A/B test waits until enough observations are collected and then uses statistical testing. ExperimentIQ adds an ML layer: it learns from user and experiment features to estimate outcomes, segment performance, and support earlier business decisions. ML should complement—not replace—proper experimental design and statistical significance testing.

Core question

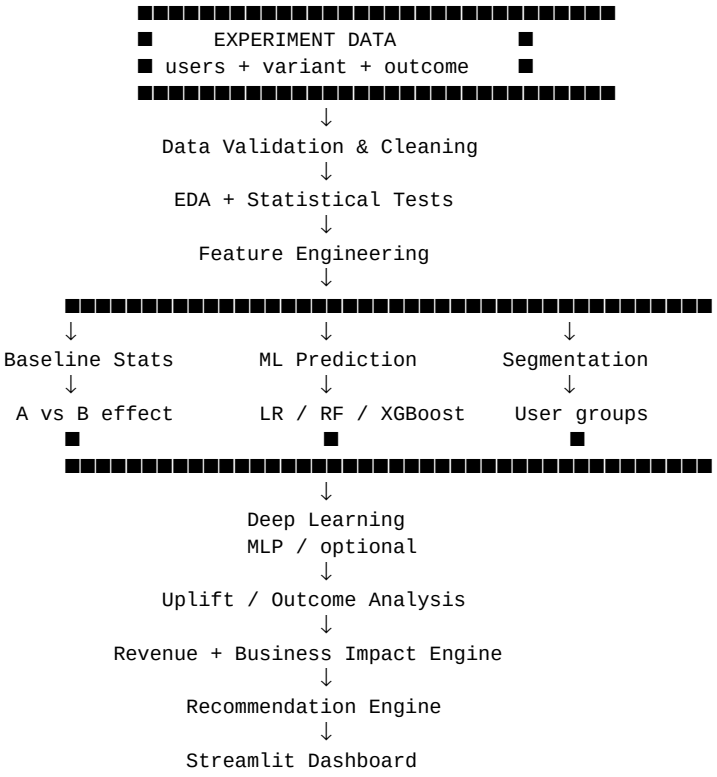
```
Given:
    User / session features
    Experiment assignment (A or B)
    Conversion outcome
    Revenue / engagement outcome

Predict:
    P(conversion | user, variant)
    Expected revenue
    Probability B outperforms A
    Segment-level uplift

Decide:
    Keep A / Deploy B / Continue experiment / Investigate
```

2. Final Product Vision

The finished system should feel like a small experimentation analytics product rather than a notebook.



Important principle

Do not claim that the model proves causality. Randomized treatment assignment provides the causal experiment; the ML model predicts outcomes and helps with segmentation, prioritization, and decision support.

3. End-to-End Workflow — Step by Step

Step 1 — Define the business experiment

Choose one measurable objective such as purchase conversion, signup, click-through rate, retention, or revenue per user. Define A, B, primary metric, guardrail metrics, experiment duration, and decision rule.

Step 2 — Collect or create data

Build a user-level dataset containing experiment assignment, user/session attributes, outcome, and optional revenue. For academic work, a synthetic or public dataset is acceptable if the assumptions are documented.

Step 3 — Data validation

Check missing values, duplicates, impossible values, leakage, treatment imbalance, inconsistent timestamps, and whether users appear multiple times in a way that violates the analysis unit.

Step 4 — Exploratory data analysis

Study conversion by variant, sample sizes, distributions, missingness, user segments, revenue distributions, and time trends.

Step 5 — Statistical baseline

Calculate conversion rate, absolute uplift, relative uplift, confidence intervals, and a suitable hypothesis test such as a two-proportion test/chi-square test. For continuous outcomes, use an appropriate t-test or non-parametric alternative.

Step 6 — Feature engineering

Create meaningful user-level features: prior activity, sessions, device, geography, traffic source, historical purchase behavior, recency/frequency features, and experiment context. Avoid features generated after treatment.

Step 7 — Build ML baselines

Start with Logistic Regression for conversion probability. Add Decision Tree/Random Forest and XGBoost. Compare models using out-of-sample evaluation.

Step 8 — Add deep learning

Build a small MLP neural network for conversion prediction. Do not add deep learning just for decoration; demonstrate whether it improves generalization over strong tabular ML baselines.

Step 9 — Estimate segment-level effects

Compare performance across meaningful groups. Optionally build an uplift-modeling component to estimate which users are more likely to benefit from B.

Step 10 — Business impact engine

Translate predicted conversion/revenue differences into expected incremental conversions, revenue, cost, and ROI.

Step 11 — Recommendation layer

Combine statistical evidence, effect size, model performance, and business impact into a transparent decision: Deploy B, Keep A, Continue Test, or Investigate.

Step 12 — Dashboard and deployment

Expose experiment metrics, model predictions, segment analysis, and recommendations through Streamlit. Optionally expose prediction endpoints with FastAPI.

4. Data Design

Recommended row structure

Column	Type	Meaning
user_id	ID	Unique user identifier
variant	A/B	Experiment treatment assignment
converted	0/1	Primary conversion outcome
revenue	Numeric	Revenue generated by user
age	Numeric	User attribute, if appropriate
device	Category	Mobile / desktop / tablet
country	Category	Geographic segment
traffic_source	Category	Organic / paid / referral etc.
sessions_before	Numeric	Historical activity before experiment
previous_orders	Numeric	Historical purchase count
days_since_last_activity	Numeric	Recency
experiment_day	Date/number	Relative experiment time

Target variables

- **Primary:** converted (binary classification).
- **Secondary:** revenue or order value (regression).
- **Advanced:** treatment uplift / individual treatment effect estimate.

Data source strategy

- Best academic route: create a realistic synthetic A/B dataset with known treatment effect so the entire pipeline is reproducible.
- Alternative: use public online experimentation/conversion datasets, while documenting what the dataset actually represents.
- Do not present synthetic results as results from a real company.

5. Statistics Module

Core metrics

Metric	Formula / idea	Purpose
Conversion rate	conversions / users	Basic performance
Absolute uplift	CR_B – CR_A	Percentage-point improvement
Relative uplift	(CR_B – CR_A) / CR_A	Relative improvement
Revenue per user	total revenue / users	Monetary outcome
Confidence interval	Estimate ± uncertainty	Quantify uncertainty
p-value	Test evidence against H0	Statistical significance

Hypotheses

Example: H0 = Variant B has the same conversion rate as A. H1 = Variant B has a different conversion rate. For a directional product decision, the hypothesis can be one-sided when justified in advance.

Decision discipline

- Statistical significance is not the same as business significance.
- A tiny statistically significant improvement may have negligible financial value.
- A large observed improvement with insufficient sample size may require continuing the experiment.
- Always inspect confidence intervals and effect size, not just p-values.

6. Machine Learning Module

Model progression

Stage	Model	Why
Baseline	Majority / mean baseline	Establish a minimum benchmark
1	Logistic Regression	Interpretable probability baseline
2	Decision Tree	Simple nonlinear relationships
3	Random Forest	Robust nonlinear tabular model
4	XGBoost	Strong tabular benchmark
5	MLP / Neural Network	Deep-learning comparison

Classification objective

Predict the probability that a user converts under the observed experimental context. Evaluate using ROC-AUC, PR-AUC, log loss, precision, recall, F1, calibration, and business-oriented metrics.

Regression objective

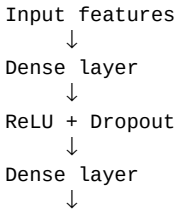
Predict revenue/order value where appropriate. Evaluate using MAE, RMSE and, when meaningful, MAPE or a transformed target. Revenue is often heavy-tailed, so inspect its distribution before choosing the loss and metric.

Model validation

- Use a proper train/validation/test split.
- Prevent treatment leakage and post-treatment features.
- If time drift is important, use a time-based split.
- Tune models only using training/validation data; keep the final test set untouched.

7. Deep Learning Extension

The deep-learning component should be an extension, not the entire project. For tabular user data, a Multi-Layer Perceptron (MLP) is a sensible academic choice.



```
ReLU + Dropout
     ↓
Output sigmoid
     ↓
P(conversion)
```

Suggested implementation

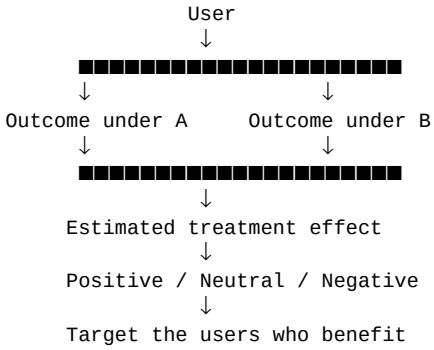
- Normalize numerical features.
- Encode categorical variables using one-hot encoding or embeddings.
- Use binary cross-entropy for conversion classification.
- Use early stopping and validation monitoring.
- Compare MLP against Logistic Regression and XGBoost.

If the MLP does not outperform XGBoost, that is a valid and useful result. It demonstrates model selection rather than blindly preferring deep learning.

8. Advanced Feature — Uplift Modeling

This is the feature that can make ExperimentIQ substantially more interesting.

Normal prediction asks: **Who will convert?** Uplift modeling asks: **Who is more likely to convert because of Variant B?**



Four useful groups

Group	Interpretation	Business action
Persuadables	B likely causes improvement	Target with B
Sure things	Likely to convert anyway	B may add little
Lost causes	Unlikely to convert either way	Do not overspend
Sleeping dogs	B may reduce conversion	Avoid B

Uplift modeling can be a future/advanced module if the core project becomes too large.

9. Business Impact Engine

The system should translate model output into business language.

```
Predicted conversion_A = pA
Predicted conversion_B = pB

Expected incremental conversions
= N × (pB − pA)

Expected incremental revenue
= incremental conversions × average order value

Estimated profit impact
```

```
= incremental revenue – implementation/marketing cost
```

Example dashboard message

```
Variant B
Conversion: 8.6%
Variant A
Conversion: 7.2%

Absolute uplift: +1.4 percentage points
Relative uplift: +19.4%

Estimated incremental conversions: 1,400 / 100,000 users
Estimated revenue impact: ■X

Statistical evidence: significant / inconclusive
Recommendation: DEPLOY B / CONTINUE TEST
```

All values above are illustrative. The final project must calculate them from your actual dataset.

10. Recommendation Engine

Use transparent rules instead of letting a black-box model make an unexplained business decision.

Condition	Recommendation
B significantly better + meaningful business impact	DEPLOY B
B not statistically conclusive + sample still small	CONTINUE TEST
B significantly worse	KEEP A
Statistically significant but tiny business impact	REVIEW COST/BENEFIT
Data quality or leakage concerns	INVESTIGATE DATA

Why this matters

This turns the project from a model demo into a decision-support system. A recruiter can see the connection between statistics, ML, and business outcomes.

11. Dashboard Design

```
■■■
```

• ML Prediction — model metrics, calibration, feature importance.

• Segment Analysis — variant performance by user segment.

• Business Impact — expected conversions/revenue/ROI.

• What-if Simulator — change traffic size, conversion uplift, or average order value.

• Model Card — data, model, assumptions, limitations.

12. Project Folder Structure

```
ExperimentIQ/
■
■■■ data/
■   ■■■ raw/
■   ■■■ processed/
■   ■■■ synthetic/
■
■■■ notebooks/
■   ■■■ 01_data_generation.ipynb
■   ■■■ 02_eda.ipynb
■   ■■■ 03_statistics.ipynb
■   ■■■ 04_ml_models.ipynb
■   ■■■ 05_deep_learning.ipynb
■   ■■■ 06_uplift_analysis.ipynb
■
■■■ src/
■   ■■■ data/
■   ■   ■■■ cleaning.py
■   ■   ■■■ validation.py
■   ■■■ features/
■   ■   ■■■ engineering.py
■   ■■■ statistics/
■   ■   ■■■ ab_tests.py
■   ■■■ models/
■   ■   ■■■ train_ml.py
■   ■   ■■■ train_xgb.py
■   ■   ■■■ train_mlp.py
■   ■■■ business/
■   ■   ■■■ impact.py
■   ■■■ utils/
■
■■■ dashboard/
■   ■■■ app.py
■
■■■ models/
■   ■■■ saved_models/
■
■■■ tests/
■■■ requirements.txt
■■■ README.md
■■■ Dockerfile
■■■ .gitignore
```

13. Technology Stack

Layer	Tools
Language	Python
Data	Pandas, NumPy
Statistics	SciPy, Statsmodels
Visualization	Matplotlib, Seaborn, Plotly
ML	Scikit-learn, XGBoost
DL	PyTorch or TensorFlow/Keras

Layer	Tools
Experiment tracking	MLflow (optional but recommended)
Dashboard	Streamlit
API	FastAPI (optional)
Database	SQLite/PostgreSQL (optional)
Deployment	Docker + a cloud platform
Version control	Git + GitHub

14. Recommended Development Roadmap

Phase	Timeline	Deliverable
Phase 1 — Problem + dataset	Day 1–3	Define experiment, metrics, schema, generate/find dataset.
Phase 2 — Data pipeline	Day 4–6	Cleaning, validation, preprocessing, feature engineering.
Phase 3 — EDA	Day 7–9	Visualize A/B performance, distributions, segments and time effects.
Phase 4 — Statistics	Day 10–12	Confidence intervals, hypothesis tests, uplift and power concepts.
Phase 5 — ML	Day 13–18	Logistic Regression, Random Forest, XGBoost, tuning and evaluation.
Phase 6 — Deep Learning	Day 19–22	MLP, regularization, early stopping, comparison.
Phase 7 — Business layer	Day 23–25	Revenue impact, ROI, recommendation engine.
Phase 8 — Dashboard	Day 26–29	Streamlit pages and interactive charts.
Phase 9 — Advanced	Day 30–34	Uplift modeling, what-if simulator, model explainability.
Phase 10 — Deployment	Day 35–38	Docker, API if needed, deployment, testing.
Phase 11 — Documentation	Day 39–42	README, architecture diagram, model card, screenshots, resume bullets.

15. Evaluation Checklist

Data

- No duplicate user records that invalidate the analysis.
- No post-treatment leakage.
- Treatment/control assignment is checked for balance.
- Missing values and outliers are documented.

Statistics

- Conversion rates and absolute/relative uplift.
- Confidence intervals.
- Appropriate hypothesis test.
- Sample-size/power discussion.

ML

- Baseline vs advanced model comparison.
- Train/validation/test separation.
- ROC-AUC + PR-AUC + log loss/calibration for classification.

- Feature importance / SHAP-style explainability if used.

Deep Learning

- MLP with validation monitoring.
- Early stopping / regularization.
- Fair comparison against XGBoost.

Business

- Expected incremental conversions.
- Expected revenue/profit impact.
- Clear recommendation with uncertainty.
- What-if analysis.

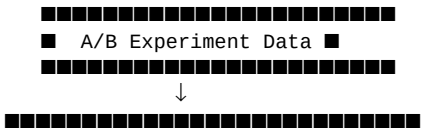
16. Common Mistakes to Avoid

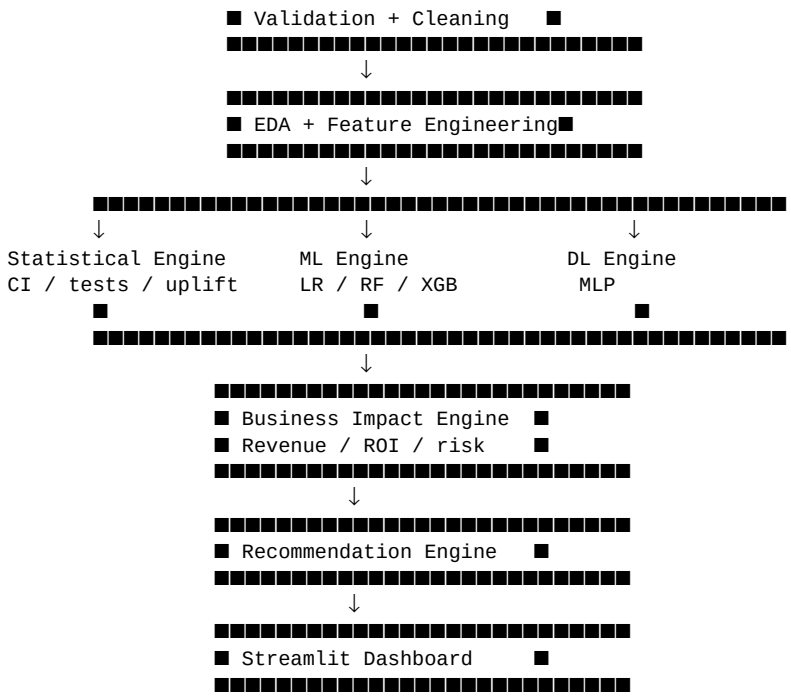
- **Do not predict the A/B winner from the variant column alone.** The model must have meaningful features and proper validation.
- **Do not use post-experiment information.** This creates leakage.
- **Do not optimize only for accuracy.** Conversion datasets are often imbalanced.
- **Do not stop an experiment simply because a p-value looks good.** Experiment design and sample-size considerations matter.
- **Do not claim causality from an observational dataset.** Clearly distinguish prediction from randomized treatment effect.
- **Do not force deep learning.** If XGBoost wins, report it honestly.
- **Do not fabricate performance numbers.** Your README and resume should contain only measured results.
- **Do not build only a notebook.** Finish with a usable dashboard and reproducible pipeline.

17. Optional Advanced Features

Feature	Difficulty	Value
MLflow experiment tracking	Medium	Shows ML engineering maturity
SHAP explanations	Medium	Model interpretability
Uplift modeling	Hard	Strong experimentation feature
What-if simulator	Medium	Excellent business demo
FastAPI prediction API	Medium	Deployment skill
PostgreSQL storage	Medium	Database engineering
Docker	Medium	Reproducible deployment
Scheduled experiment reports	Medium	Production-style workflow

18. Final Architecture





19. Resume Positioning

Once the project is actually completed and the metrics are measured, a strong resume section could look like:

```
ExperimentIQ | ML-Powered A/B Testing & Business Experimentation
• Built an end-to-end experimentation platform combining statistical
  hypothesis testing with Logistic Regression, Random Forest, XGBoost
  and a neural network to predict conversion outcomes.
• Developed business-impact analytics to estimate incremental
  conversions, revenue uplift and experiment ROI across user segments.
• Deployed an interactive Streamlit dashboard for experiment analysis,
  model evaluation, segment-level insights and automated recommendations.
```

Replace every performance claim with your actual measured result. For example, if your XGBoost model achieves a particular PR-AUC or your simulator produces a particular measured improvement, use that real number.

20. What You Will Learn

Area	Skills gained
Statistics	Hypothesis testing, confidence intervals, power, effect size
ML	Classification, regression, ensembles, tuning, calibration
Deep Learning	Tabular neural networks, regularization, training
Feature Engineering	User behavior and historical features
Experimentation	A/B testing and treatment effects
Business	Revenue impact, ROI, decision rules
MLOps	Tracking, testing, model versioning
Deployment	Streamlit, APIs, Docker
Communication	Turning model output into business decisions

21. Final Build Order — The Short Version

```
1. Define business problem
2. Create/select dataset
```

```
3. Build data validation pipeline
4. Perform EDA
5. Implement A/B statistical analysis
6. Engineer features
7. Train Logistic Regression
8. Train Random Forest
9. Train XGBoost
10. Evaluate + tune
11. Train MLP neural network
12. Compare ML vs DL
13. Add segment/uplift analysis
14. Build revenue/ROI engine
15. Build recommendation engine
16. Build Streamlit dashboard
17. Add MLflow / FastAPI / Docker
18. Test everything
19. Write README + architecture
20. Deploy and add real screenshots/results to resume
```

22. Final Goal

The finished ExperimentIQ project should answer five questions clearly:

- Did B outperform A?
- How certain are we?
- Which users/segments benefit most?
- What is the expected business impact?
- What should the company do next?

Recommended scope: Build the core project first (statistics + Logistic Regression + Random Forest + XGBoost + MLP + business dashboard). Add uplift modeling, FastAPI, MLflow, PostgreSQL and Docker only after the core pipeline is reliable.